

Chat Application: A distributed-based approach to implement chat application

Group 06

Ahmed Mahfouz: 3500154; Hazem Ibrahim: 3574588; Mohamed Altaweel: 3280623;

Institute of Architecture of Application Systems

1 Introduction

Nowadays, most systems need to be physically distributed and deployed on several nodes. These units should communicate with each other to fulfill the software requirements and achieve the minimum features such as synchronization, availability, and dynamic resiliency. In this project, we intend to develop a chat application, known also as an instant messaging system. The software's primary goal is to provide users with an instant messaging tool that can handle millions of users simultaneously when needed and is simple to use. The application is intended to be used as a distributed application. It provides a general architecture for chat applications, and it can be used as the foundation for providing instant messaging capabilities by anyone or any organization. The project provides the back-end functionality for a distributed chat program which can be integrated further with some interfaces as a web, desktop, or mobile application.

2 Project Requirements

We start with the basic requirements for a basic chat application, which could be further extended.

- **R1:** As a user, I want to be connected to several participants in a chat one room.
- **R2:** Each user could send and receive messages synchronously.
- **R3:** As a user, I want to enter an existing chat room without restarting the room configuration
- **R4:** As a room admin, I want to be able to accept new chat participating requests.
- **R5:** As a group of users, we want to preserve the order of the message with a casual consistency.
 - Each connected user should be able to deliver its message.
 - messages should be delivered in FIFO order.
 - messages which could not be instantly delivered should be delivered lately, regardless of its actual timestamp.
- **R6:** As a user, I want to be informed if the server component fails and the system can not deliver messages anymore.

3 System Architecture

To reduce the complexity of communication and be able to organize the chatting process, a client-server architecture is implemented in this project. peer-to-peer communication is much complex and has to send the same message for all participants and implement a local synchronization on each component to preserve a casual order of messages.

A server is a component that listens for incoming requests from users. The server is responsible for accepting new incoming requests, adding them to an existing chat group, or creating a new group for them. The client is an application deployed on the end-user device to communicate with the server, send requests, and receive incoming responses.

The server instances should be seen as one image to all the clients, as shown in figure 1. Thus, a client should send an incoming request without knowing the instance is responsible for the desired group. Thus, the leader server should elect an instance to take the full responsibility of handling the incoming request, whether it's a new group creation or joining an existing group (which in this case should be automatically assigned to the elected responsible instance) each instance could be responsible for one or even more chat groups.

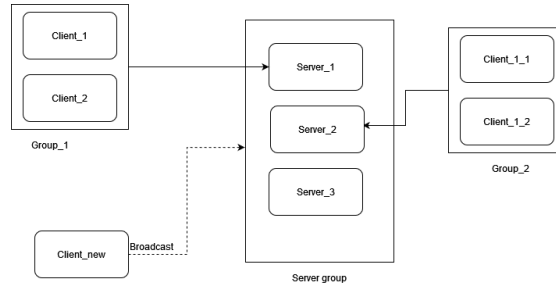


Fig. 1. The communication between a group of different clients and the server

4 Application relevant features

In this section, we explain how we implemented the important concepts in distributed systems design for each use case in our chat application.

4.1 Dynamic discovery of hosts

In order to enable a dynamical Discover of which nodes are currently connected to the system, we use broadcasting between server nodes. Broadcasting means

sending a message one time to a specific address, in which all relevant nodes are listening to. Most cases as we will see in our application of broadcasting is declaring announcements such as a new node's join or a node is crashed or leaving the group.

Dynamic discovery intend to improve the dynamism of the system, especially when new components join the system. As an example from our case, the client might know only one IP address of a server (further, it could be a domain name or a broadcasting address). When a new client comes, the server broadcast all other servers with the new client and respond with the selected server for handling the connection with the client so that it is connected successfully as shown in the workflow figure 2.

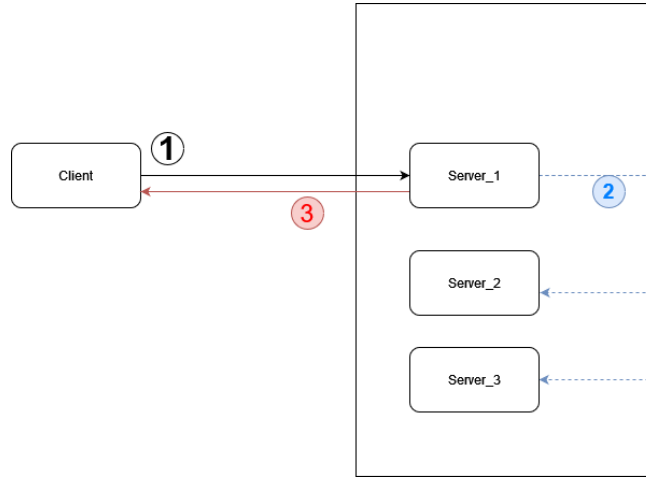


Fig. 2. Main steps when a client connect to the system

Another case when a new server is started and should join the system. Here we need to prevent the restart of the whole servers instances currently running. Hence, each new server instance should broadcast a message requesting a join, Then, the leader server would accept the connection and notify all other instances - including the new one - of the updated list of servers.

In addition to new participants, each server nodes leaves the system (or might crash, as we see later) should announce it so that other nodes can update its nodes list. It is also important for the leader server to have an updated list of all available nodes to assign a client to it correctly.

In summary, broadcasting technique helps to maintain a solid synchronized state of all server nodes to preserve the concept of one representative image, that could respond and handle external requests as a one atomic point.

4.2 Multicasting

As shown in figure 1, When a new client enters the chat application, it has the option to join an existing chat room or to create a new chat a room. If it chooses to enter an existing chat room, the server that is responsible to for handling this chat room updates its group view by adding the new client to its clients list and broadcast the server-groups list to the other servers. and the new client stats multicast his messages to all the clients of its own groups. If the new client chooses to create a new chat room, a new server takes the responsibility of handling the new chat room, create a new client lists and broadcast it to the other servers. Each chat room is assigned to one server node, so if a client belongs to the chat room sends a message, each client of the same room can receive the message via a multicasting process in which the server select only the clients of it's the group to share messages between them not with clients in another chat room. The server goes through all the connected client sockets except the sender and send them the message.

4.3 Leader election

The goal of leader election is to grant specific powers to one item (a process, host, thread, object, or human) in a distributed system. These specific abilities could include the ability to delegate tasks, edit data, or even be in charge of managing all system requests.

In our case, the server leader should handle the new incoming requests, whether from client or server nodes. When a new client joins, then it selects a server and forward the connection to it. Currently, the selection is done randomly but could be integrated with a load-balancing algorithm since it has information about the current servers, with their assigned groups from which it can retrieve the number of connected clients.

A further important case - which is currently not finished in implementation - is the health checking of other nodes. Some nodes might face network problem, so they can not even reach a failure case or the announcement message is lost. For this reason, it is also important that the leader server to check the status of other nodes periodically. This can be easily done with broadcasting. The algorithm 1 explain the steps in which the leader announces a checking message, starting a timeout and when a node is not responding, then it will consider it out of the system and inform all others with the deletion update.

Algorithm 1 Health Check

```

Broadcast(peep-check)
for a doll server nodes listed
    Wait for response
    if timeout then
        remove from servers list
    end if
end for
Broadcast(servers-list)

```

When a server joins the system, it waits for a response if there is already a node is running. If the server does not get any response within a time limit, then it validates that it is the first server of the system and assigns itself as a leader. However, when a node is already running, then the response contains the current leader IP address. In this case, we do not have to modify or decide if a node should start a leader or not, but the system perform the decision automatically.

When a new leader should be elected, a node could trigger the election process. First, it notifies all other nodes in order to open a socket for sending and listening the election messages. The election is based on LaLenn-Cheng-Roberts Algorithm. Hence, the election trigger sends a list of the server's IP addresses. It aims to ensure the correct flow of the algorithm, so each node gets the same list of members participated in the election (one source of truth) and forms a ring from it. The new elected node broadcast itself as a new leader, so all other nodes ends the process of election. The goal of this announcement is to optimize the performance of the system by using fewer threads and resources(memory, ports) only when needed and reactivate it again when a new election is triggered. we summarized both workflows in listing 3 & 2.

To sum up, the leading feature has a crucial role of the coordination between the different parts of the system. A leader is the responsible for the critical changes might affect the system. Therefore, the election process need to be robust and can always deliver a valid result. Currently, the election could be triggered manually from the user and later on in case of leader failure. In this way, the system will be able to integrate with other mechanism and event handler which start the election when it is required.

Algorithm 2 Election trigger

```

1: members = generateAlistOfServers()
2: Broadcast(start-election, members)
3: FormAring(members)
4: markItselfAs participant
5: electionMessage = mid:self.id, isLeader:self.isLeader
6: sendMessageOfAring(left,electionmessage )

```

Algorithm 3 Election process

```

1: Open election socket
2: if electionMessage mid false then
3:   if mid < pid and not participant then
4:     send (pid, false)
5:     participant = True
6:   end if
7:   if mid > pid and not participant then
8:     send (mid, false)
9:     participant = True
10:  end if
11:  if mid == pid then
12:    declare itself as a leader
13:    not a participant
14:  end if
15: end if
16: if electionMessage mid true and participant then
17:   send (mid,true)
18:   not a participant
19: end if

```

4.4 Fault Tolerance

Fault tolerance is a critical issue in distributed computing; it keeps the system operational in the event of a failure. The most important aspect of it is to keep the system running even if any of its components fail. It is related to dependable systems for a system to be fault-tolerant. Dependability encompasses several useful requirements in the fault tolerance system, including Availability, Reliability, Safety, and Maintainability. In our case, we aim to implement a reliable system with the highest possible level of availability.

The first critical case is the cut-off connection between a client and a server node. regardless of the node type and since the system does not save the chat messages, it is easier to forward the connection to another node. The figure 3 explain the process steps. When the client except a cutoff error, it broadcast a message again. The server leader will check the room name again. If the node still exists, then it will resend the IP address again to the client and try to reconnect to it. But if the node went down, then the leader will assign a new node and send its IP to the client to connect to it.

The second important case is how the system deals when a node leaves the system or crashes. When a normal server node exits the system, it declares its leave through broadcasting, and each node maintains the server list. The problem is when a leader announce its leave. In this case, we should trigger an election process to select a new leader. The leader server should here not participate since it is leaving. Hence, we modified the LCR algorithm initialization 4. The leader declares an election start, sending a list of all servers without it. In order

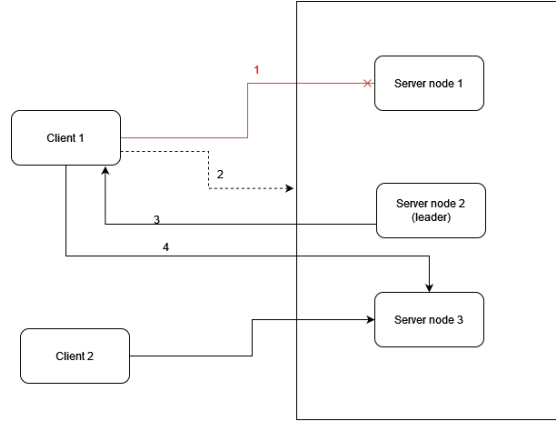


Fig. 3. The client reconnect to the system and a new server node

to prevent being elected again, the leader sends a zero ID instead of its own ID, so each node can win the election over it. Further, the nodes form a ring without the leader and elect a new one. This ring-break is shown in figure 4, where the leader breaks the ring to leave it, and its right neighbor connect its left one.

Algorithm 4 Election trigger

```

1: members = generateAlistOfServers() - self
2: Broadcast(start-election, members)
3: FormAring(members + self )
4: markItselfAs participant
5: electionMessage = mid:0, isLeader:self.isLeader
6: sendMessageOfAring(left,electionmessage )

```

5 Implementation

In this section, we describe the code on a very high level to explain the general structure of it. The project is found in [GitHub Project-click-here](#).

The ultimate goal of this project is to implement the backend of chat application system. Hence, we split the logic in a component and integrate it currently with a simple console interface, in which we can start, exit, and pass the required arguments to the business logic, that implements a distributed system.

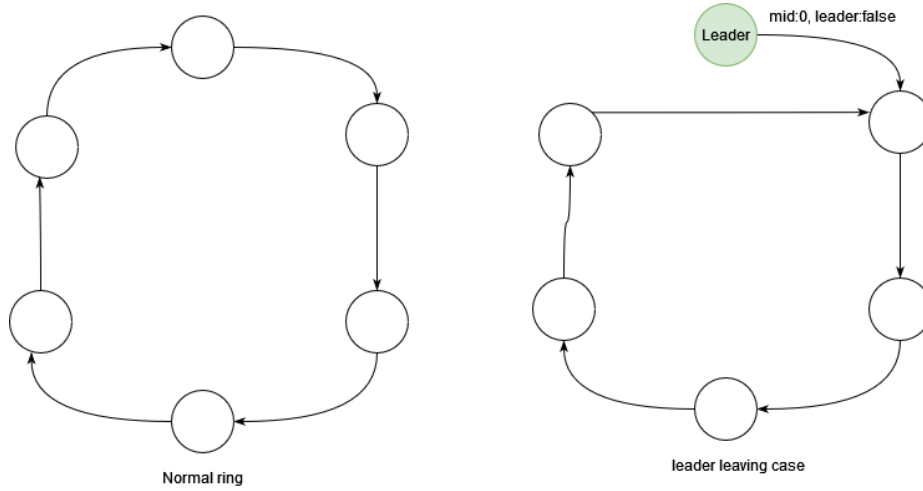


Fig. 4. Difference between the normal ring and when leader is leaving the ring

5.1 Server Component

The server component "server.py" class file is the one responsible for receiving incoming requests from the users, handle it correctly, and forward the response. A server component includes the following functionality.

- Broadcast messages to other servers.
- Handle broadcasted messages from other servers.
- Accept incoming connections from client.
- Multicast a message from a client to a specific group of clients.
- Start and trigger an election to choose a leader server node.
- Save and maintain a list of server nodes with its assigned chat rooms.

The following ports are reserved and fixed in each node:

- 3795 for broadcasting.
- 10001 for election process communication.

5.2 Client component

The client component "client.py" is the one installed on the user side, which sends messages and requests to the servers. A client do the following tasks:

- Broadcast a join request to the server.
- Send messages to a group of clients.

6 Discussion & Limitation

The system is still under development. Many features could be integrated in the future to make the system more robust. Some of the important points we consider:

- Implementing a message queue to deal more with breaking connection with clients. One example is that the system would be able to deliver all messages if a client went offline for a while and reconnects back.
- Implementing health check functions which could be added to the leader to execute periodically.
- handles connection problems during election which might break the closed ring.

7 Conclusion

In this report, we introduced a chat distributed application to exchange messages between different nodes. we provide a general architecture for backend chat applications, which can be deployed further with some interfaces as a web, desktop, or mobile application. We introduced the basic and important concepts in distributed system designing to fulfill our requirements. The system designed in general as an extensible tool which can be further extended including new functionalities or with other libraries or frameworks.